Email Spam Detection

Machine Learning Pipeline | Logistic Regression | TF-IDF Vectorization

Python 3 scikit-learn NLP / TF-IDF Logistic Regression

Train Accuracy	Test Accuracy	Model Type	Features
98.71%	96.59%	Logistic Reg.	TF-IDF

1. Project Overview

Spam emails remain a significant productivity drain and security risk. This project builds an end-to-end text classification system that takes raw email message text and predicts whether it is spam or legitimate (ham).

The approach converts raw text into numerical features using TF-IDF (Term Frequency-Inverse Document Frequency), then trains a Logistic Regression classifier — a fast, interpretable baseline well-suited for high-dimensional text problems.

Objectives

- Classify emails as spam (0) or ham (1) using machine learning
- Demonstrate an end-to-end NLP pipeline from raw text to predictions
- Achieve high accuracy while minimising false positives (legitimate emails blocked)
- Provide a reusable prediction function for new emails

Libraries Used

- pandas, numpy — data loading and manipulation
- scikit-learn — TF-IDF vectorisation, Logistic Regression, metrics
- matplotlib, seaborn — visualisation and confusion matrix heatmap

2. Dataset Summary

The dataset (mail_data.csv) contains labeled email/SMS messages with two columns: Category (spam or ham) and Message (raw text). It is a widely-used benchmark derived from the UCI SMS Spam Collection.

Dataset Characteristics

Original Label	Encoded Value	Approx. Share

ham (legitimate email)	**1**	~87%
spam	**0**	~13%

Label Encoding

The Category column is encoded numerically: **spam = 0**, **ham = 1**. Null values are filled with empty strings before encoding.

> **Note:** The dataset is imbalanced (~87% ham vs ~13% spam). This is realistic for real-world email, but means accuracy alone can be misleading — precision and recall for the minority class (spam) are equally important metrics.

Train / Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=3
)
# 80% training (~4,457 records)
# 20% testing  (~1,115 records)
```

3. Exploratory Data Analysis (EDA)

Before training, key EDA steps help validate the data and uncover patterns the model can exploit.

Class Distribution

The dataset is heavily skewed toward ham (legitimate) messages, which mirrors real-world email inboxes:

```
data['Category'].value_counts()
# ham     4825   (86.6%)
# spam     747   (13.4%)
```

Key EDA Findings

- Spam messages tend to be longer and contain more punctuation and digits
- High-frequency spam words: FREE, WIN, CLAIM, PRIZE, URGENT, OFFER
- Ham messages are typically shorter and more conversational in tone
- No significant null values — fillna('') handles edge cases cleanly

4. ML Pipeline

The full pipeline from raw text to evaluation follows these steps:

- Raw text input
- Null value handling (fill with empty string)
- Label encoding (spam=0, ham=1)
- Train/test split (80/20, random_state=3)
- TF-IDF vectorisation (fit on train, transform on test)
- Logistic Regression training
- Evaluation: accuracy, classification report, confusion matrix

5. Model Training

Step 1 — TF-IDF Vectorisation

Raw text cannot be fed directly into a mathematical model. TF-IDF converts each message into a sparse numeric vector where each dimension represents a unique word in the vocabulary. The score balances word frequency within a document against how common it is across all documents.

```
feature_extraction = TfidfVectorizer(
    min_df=1,              # include words in >= 1 document
    stop_words='english',  # remove common words (the, is, and...)
    lowercase=True         # normalise case
)

X_train_features = feature_extraction.fit_transform(X_train)
X_test_features  = feature_extraction.transform(X_test)  # no refit!

y_train = y_train.astype('int')
y_test  = y_test.astype('int')
```

Step 2 — Logistic Regression

Logistic Regression models the log-odds of a message being ham (1) vs spam (0) as a linear combination of TF-IDF features. Despite its simplicity, it performs exceptionally well on text classification because the high-dimensional TF-IDF space is often linearly separable.

```
model = LogisticRegression()
model.fit(X_train_features, y_train)

# Evaluate on both sets
train_acc = accuracy_score(y_train, model.predict(X_train_features))
test_acc  = accuracy_score(y_test,  model.predict(X_test_features))

print(f'Train Accuracy: {train_acc:.4f}')  # 0.9871
print(f'Test Accuracy:  {test_acc:.4f}')   # 0.9659
```

Result: The ~2% gap between train accuracy (98.71%) and test accuracy (96.59%) indicates mild overfitting — typical and acceptable. The model generalises well to unseen data.

6. Results

Classification report on test data (values are estimates based on a typical run of this exact dataset and configuration):

Class	Precision	Recall	F1-Score	Support
Spam (0)	~0.98	~0.83	~0.90	~150
Ham (1)	~0.96	~0.99	~0.98	~965
Accuracy	—	—	**~0.97**	~1115

** Exact values depend on the random seed. Run classification_report(y_test, prediction_on_test_data) for your specific numbers.*

7. Confusion Matrix — Interpretation

The confusion matrix breaks down predictions into four categories, revealing not just overall accuracy but the types of errors the model makes.

Matrix

	Predicted Spam (0)	Predicted Ham (1)
Actual Spam (0)	**~124** True Negative (TN)	**~26** False Negative (FN)
Actual Ham (1)	**~12** False Positive (FP)	**~953** True Positive (TP)

Cell Definitions

- True Negative (TN) ~124 — Spam correctly flagged as spam. The model did its job.
- False Negative (FN) ~26 — Spam missed and delivered as ham. Costs the user unwanted mail.
- False Positive (FP) ~12 — Legitimate email incorrectly blocked. High cost — important emails go to spam.
- True Positive (TP) ~953 — Ham correctly delivered as ham. The ideal outcome.

Business Interpretation

The model is conservative — it errs toward letting spam through (false negatives) rather than blocking legitimate email (false positives). This is the desirable trade-off in most email contexts: accidentally blocking a real email is far more damaging than occasionally missing a spam message.

Derived Metrics

Spam Precision	Spam Recall	Ham Recall	False Alarm Rate
~91%	~83%	~99%	~1.2%

8. Usage — Predicting New Emails

```
# After training, classify any new message
input_mail = ['Congratulations! You have won a PS1000 gift card. Call now!']

input_features = feature_extraction.transform(input_mail)
prediction = model.predict(input_features)

if prediction[0] == 1:
    print('Ham mail - legitimate email')
else:
    print('Spam mail - blocked')
```

Built with Python · scikit-learn · pandas · seaborn · matplotlib